

**ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅԱՆ ԳԻՏՈՒԹՅԱՆ և ԿՐԹՈՒԹՅԱՆ
ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ ՀԱՅԱՍՏԱՆԻ ՊԵՏԱԿԱՆ ՃԱՐՏԱՐԱԳԻՏԱԿԱՆ
ՀԱՄԱԼՍԱՐԱՆ**

ՍԻՆՈՓՍԻՍ ԱՐՄԵԻՆԱ ԸՆԿԵՐՈՒԹՅԱՆ ԴԵՊԱՐՏԱՄԵՆՏ



SYNOPSYS®

ՏՀՏԷ ինստիտուտի ՄՍ և Հ ամբիոն SS019-Ս խումբ

Ծրագրավորման C++ լեզու

«Տիեզերական մրցաշար» («Space Race»)

Ուսանող՝ Ալվարդ Բաբայան

Ղեկավար՝ Ռաֆայել Շահոյան

ԵՐԱԱՆ 2022

Բովանդակություն

Խնդրի դրվածքը -----	3
Աշխատանքի կառուցվածքը -----	3
Խաղի կանոնները -----	4
Խաղի ճարտարապետությունը -----	5
Խաղի ծրագիրը -----	7
Արտաքին դիզայնը -----	34
Եզրակացություն -----	36
Գրականության ցանկ -----	36

Խնդրի դրվածքը

1. Իրագործել «**Տիեզերական մրցաշար**»(**«Space Race»**) խաղի կոնսոլային տարբերակը:
2. Իրագործել սկզբնական **Menu**, որն էլ իր հերթին ունի 4 հիմնական դաշտեր՝ **Play, Options, Records, Quit**:
3. Աշխատանքը կատարելիս օգտվել C++ ծրագրավորման լեզվի **ncurses** գրադարանից օգտագործողի միջավայրը ավելի հարմարավետ դարձնելու համար:

Աշխատանքի կառուցվածքը

Խաղը ունի **Menu**`

- Play
- Options
- Records
- Quit

Play-ը իր հերթին ունի հետևյալ դաշտերը`

- Sign in
- Back to menu

Options-ը ունի հետևյալ դաշտերը`

- 60 seconds
- 120 seconds

Records-ում խաղացողների ցուցակն է գտնվում:

Quit դաշտը ապահովում է խաղից դուրս գալու պրոցեսը:

Խաղի կանոնները

Menu դաշտում ցանկացած ընտրություն կատարելու համար պետք է սեղմել **ENTER** կոճակը:

Options դաշտից խաղացողները կարող են ընտրել իրենց նախընտրած տևողությունը խաղի համար՝ 60 վայրկյան կամ 120 վայրկյան:

Records-ում գրանցվում են խաղացողների անունները և միավորները:

Sign in-ում գրանցվում են երկու խաղացողները:

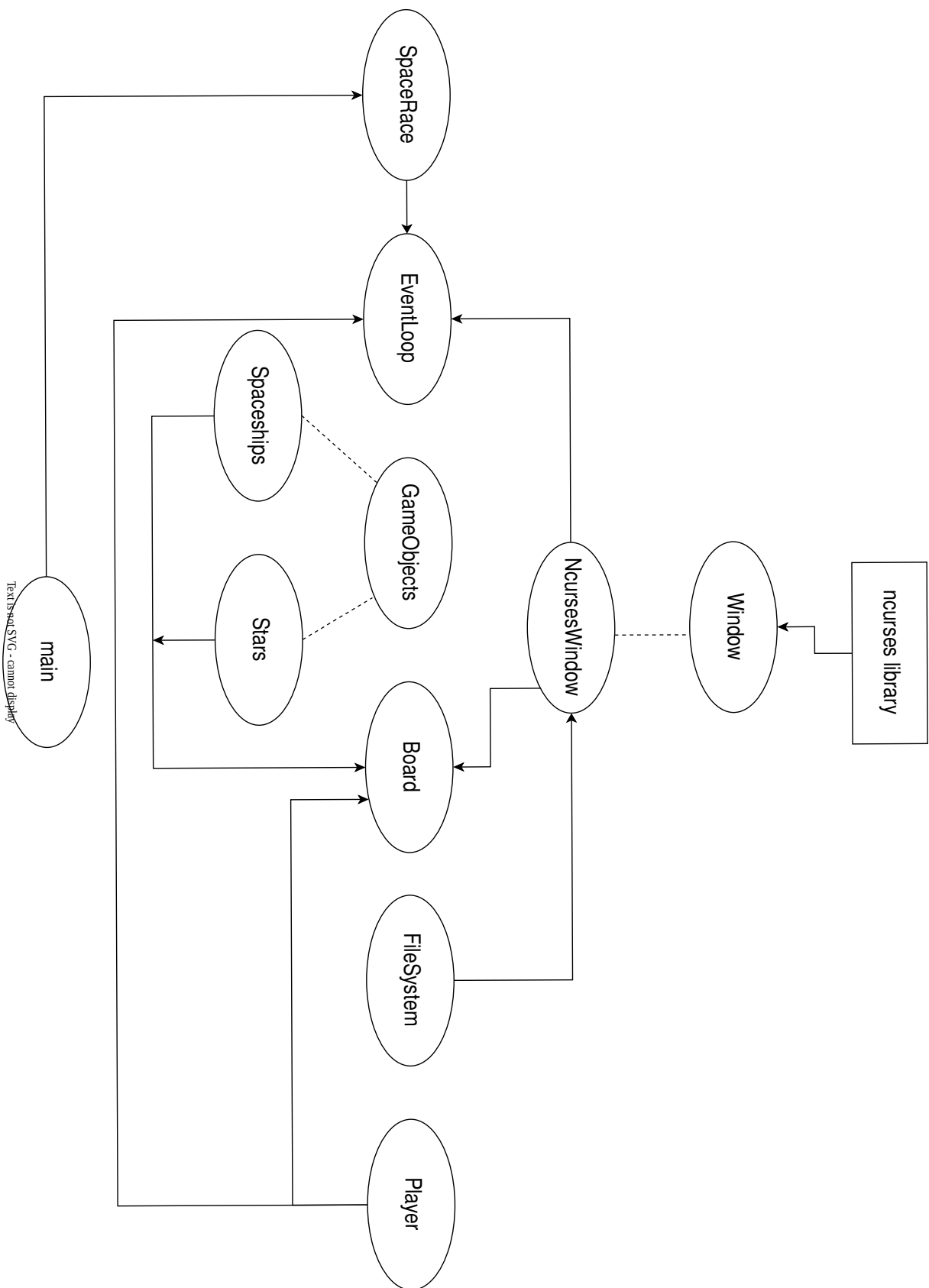
Առաջին խաղացողը խաղում է **↑** և **↓** սլաքներով, համապատասխանաբար **վերև** և **ներքև** շարժելու համար իր տիեզերանավը:

Երկրորդը՝ W և **S** տառերով, համապատասխանաբար **վերև** և **ներքև** շարժելու համար իր տիեզերանավը:

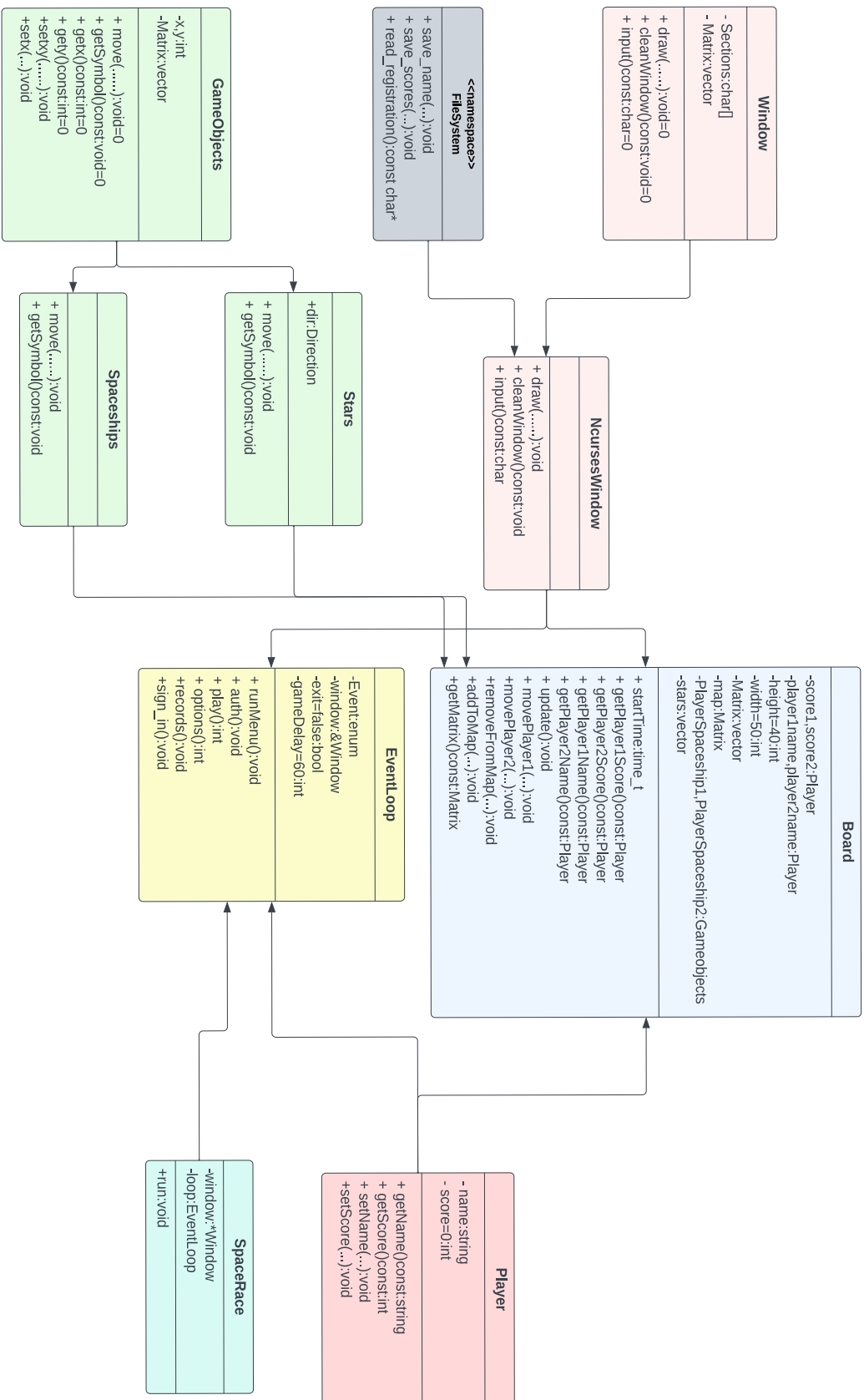
Quit դաշտը ընտրելով խաղից դուրս կգա խաղացողը:

Ցանկացած պահի խաղից դուրս գալու համար խաղացողը պետք է սեղմի **ESC** կոճակը:

Խաղի ճարտարապետությունը



Խաղի ճարտարապետությունը



Խաղի ծրագիրը

//main.cpp

```
#include "SpaceRace/SpaceRace.hpp"

int main()
{
    SpaceRace game;
    game.run();
    return 0;
}
```

//SpaceRace.hpp

```
#ifndef SPACE_RACE_HPP
#define SPACE_RACE_HPP

#include "Window/NcursesWindow.hpp"
#include "EventLoop/EventLoop.hpp"

class SpaceRace
{
private:
    Window *window;
    EventLoop loop;
public:
    SpaceRace();
    ~SpaceRace();
    void run();
};

#endif // SPACE_RACE_HPP
```

//SpaceRace.cpp

```
#include "SpaceRace.hpp"
```

```

SpaceRace::SpaceRace() : window(new NcursesWindow()), loop(window) {}

SpaceRace::~SpaceRace()
{
    delete window;
}

void SpaceRace::run()
{
    loop.runMenu();
}

```

//Window.hpp

```

#ifndef WINDOW_HPP
#define WINDOW_HPP

#include <string>
#include <vector>

class Window
{
protected:
    char menuSections[4][10] = {"PLAY", "OPTIONS", "RECORDS", "QUIT"};
    char menuItem[10];
    char authSections[2][13] = {"SIGN_IN", "BACK_TO_MENU"};
    char authItem[20];
    char timeSets[2][15] = {"60 SECONDS", "120 SECONDS"};
    char timeItem[15];
    using Matrix = std::vector<std::vector<char>>>;
public:
    Window() {}
    virtual ~Window() {}
    virtual void drawTime(int x, int y, int seconds) const = 0;
    virtual void drawScore(int x, int y, int scores) const = 0;
    virtual void drawMenu(int select) = 0;
}

```



```

virtual void drawAuth(int select) = 0;
virtual void drawTimeSet(int select) = 0;
virtual void drawText(int x, int y, std::string text) const = 0;
virtual void drawBoard(const Matrix &) const = 0;
virtual void cleanWindow() const = 0;
virtual char input() const = 0;
};

#endif // WINDOW_HPP

```

//NcursesWindow.hpp

```

#ifndef NCRUSES_WINDOW_HPP
#define NCRUSES_WINDOW_HPP

#include "Window.hpp"
#include <ncurses.h>

class NcursesWindow : public Window
{
public:
    NcursesWindow();
    ~NcursesWindow();

    void drawTime(int x, int y, int seconds) const;
    void drawScore(int x, int y, int scores) const;
    void drawMenu(int select);
    void drawAuth(int select);
    void drawTimeSet(int select);
    void drawText(int x, int y, std::string text) const;
    void drawBoard(const Matrix &) const;
    void cleanWindow() const;
    char input() const;
};

#endif // NCRUSES_WINDOW_HPP

```

//NcursesWindow.cpp

```

#include "NcursesWindow.hpp"

#include "../Utils/FileSystem/FileSystem.hpp"

#include <ncurses.h>

NcursesWindow::NcursesWindow()
{
    initscr();

    keypad(stdscr, true);

    noecho();

    halfdelay(2);

    curs_set(0);
}

NcursesWindow::~NcursesWindow()
{
    endwin();
}

void NcursesWindow::drawMenu(int select)
{
    for (int i = 0; i < 4; i++)
    {
        if (i == select)
            attron(A_STANDOUT);
        else
            attroff(A_STANDOUT);
        sprintf(menuItem, "%-7s", menuSections[i]);
        mvprintw(25 + i + 1, 70 + 2, "%s", menuItem);
    }
}

void NcursesWindow::drawAuth(int select)
{
    for (int i = 0; i < 2; i++)

```

```

{
if (i == select)
{
attron(A_STANDOUT);
}
else
{
attroff(A_STANDOUT);
}
sprintf(authItem, "%-7s", authSections[i]);
mvprintw(25 + i + 1, 70 + 2, "%s", authItem);
}
}

void NcursesWindow::drawTimeSet(int select)
{
for (int i = 0; i < 2; i++)
{
if (i == select)
{
attron(A_STANDOUT);
}
else
{
attroff(A_STANDOUT);
}
sprintf(timeItem, "%-7s", timeSets[i]);
mvprintw(25 + i + 1, 70 + 2, "%s", timeItem);
}
}

void NcursesWindow::drawText(int x, int y, std::string text) const

```

```

{
mvprintw(x, y, "%s", text.c_str());
}

void NcursesWindow::drawBoard(const Matrix &map) const
{
for (int i = 0; i < (int)map.size(); ++i)
{
move(i + 5, 5);
for (int j = 0; j < (int)map[i].size(); ++j)
{
if (j == (int)map[0].size() / 2)
{
printw("|");
}
if (map[i][j] == '^')
printw("|%c|", map[i][j]);
else
printw(" %c ", map[i][j]);
}
}
refresh();
}

void NcursesWindow::drawTime(int x, int y, int seconds) const
{
const int min = seconds / 60;
const int sec = seconds - (min * 60);
mvprintw(x, y, "Timer [%02d:%02d]", min, sec);
}

void NcursesWindow::drawScore(int x, int y, int scores) const
{

```

```

mvprintw(x, y, "%s", FileSystem::convert_int_to_char(scores));
}

char NcursesWindow::input() const
{
return getch();
}

void NcursesWindow::cleanWindow() const
{
clear();
}

```

//Board.hpp

```

#ifndef BOARD_HPP
#define BOARD_HPP

#include "GameObjects.hpp"
#include "../Player/Player.hpp"
#include <time.h>
#include <string>

class Board
{
private:
Player score1;
Player score2;
Player player1;
Player player2;
int height = 40;
int width = 50;

using Matrix = std::vector<std::vector<char>>>;
Matrix map = Matrix(height, std::vector<char>(width, ' '));
GameObjects *PlayerSpaceShip1;
GameObjects *PlayerSpaceShip2;

```

```

std::vector<GameObjects *> stars;

public:
Board(Player player1, Player player2);
~Board();

time_t startTime;

Player getPlayer1Score() const;
Player getPlayer2Score() const;

Player getPlayer1() const;
Player getPlayer2() const;

void update();

void movePlayer1(Direction);
void movePlayer2(Direction);

void removeFromMap(GameObjects *);
void addToMap(GameObjects *);

Matrix getMatrix() const;
};

```

```

#endif // BOARD_HPP

```

//Board.cpp

```

#include "Board.hpp"
#include "Spaceships.hpp"
#include "Stars.hpp"
#include <stdlib.h>

Board::Board(Player player1, Player player2) : player1(player1), player2(player2)
{
    startTime = time(0);

    PlayerSpaceShip1 = new Spaceships();
    PlayerSpaceShip1->setxy(height - 1, width / 4);

    PlayerSpaceShip2 = new Spaceships();
    PlayerSpaceShip2->setxy(height - 1, width - (width / 4));

    addToMap(PlayerSpaceShip1);
}

```

```

addToMap(PlayerSpaceShip2);

srand(time(0));

Direction dir;

for (int i = height - 3; i > 0; i -= 2)
{
    if (rand() % 2 == 0)
        dir = Direction::Right;
    else
        dir = Direction::Left;
    auto star = new Stars(dir);
    star->setxy(i, rand() % (int)(width - 1));
    stars.push_back(star);
}

Board::~Board()
{
    delete PlayerSpaceShip1;
    delete PlayerSpaceShip2;
    for (auto &star : stars)
        delete star;
    stars.clear();
}

Player Board::getPlayer1Score() const
{
    return score1;
}

Player Board::getPlayer2Score() const
{
    return score2;
}

```

```

void Board ::movePlayer1(Direction dir)
{
    int score1=getPlayer1().getScore();
    removeFromMap(PlayerSpaceShip1);
    PlayerSpaceShip1->move(dir, map);
    if (PlayerSpaceShip1->getx() == 0)
    {
        ++score1;
        player1.setScore(score1);
        addToMap(PlayerSpaceShip1);
    }
    addToMap(PlayerSpaceShip1);
}

void Board ::movePlayer2(Direction dir)
{
    int score2=getPlayer2().getScore();
    removeFromMap(PlayerSpaceShip2);
    PlayerSpaceShip2->move(dir, map);
    if (PlayerSpaceShip2->getx() == 0)
    {
        ++score2;
        player2.setScore(score2);
        addToMap(PlayerSpaceShip2);
    }
    addToMap(PlayerSpaceShip2);
}

void Board ::removeFromMap(GameObjects *obj)
{
    const int x = obj->getx();
    const int y = obj->gety();

```



```

map[x][y] = ' ';
}

void Board ::addToMap(GameObjects *obj)

{
const int x = obj->getX();
const int y = obj->getY();
map[x][y] = obj->getSymbol();
}

Board::Matrix Board::getMatrix() const
{
return map;
}

void Board::update()
{
for (int i = 0; i < (int)stars.size(); ++i)
{
removeFromMap(stars[i]);
stars[i]->move(Direction::None, map);
int x = stars[i]->getX();
int y = stars[i]->getY();
if (map[x][y] == '^')
{
if (y <= width / 2)
{
removeFromMap(PlayerSpaceShip1);
PlayerSpaceShip1->setx(height - 1);
addToMap(PlayerSpaceShip1);
}
else
{

```

```

removeFromMap(PlayerSpaceShip2);
PlayerSpaceShip2->setx(height - 1);
addToMap(PlayerSpaceShip2);
}
}
addToMap(stars[i]);
}
}

Player Board::getPlayer1() const
{
return player1;
}

Player Board::getPlayer2() const
{
return player2;
}

```

//EventLoop.hpp

```

#ifndef EVENT_LOOP_HPP
#define EVENT_LOOP_HPP

#include "../Window/Window.hpp"
#include "../Player/Player.hpp"

class EventLoop
{
private:
Window *window;

bool exit = false;

int gameDelay=60;

enum Event
{
Key_Up = 3,

```

```

Key_Down = 2,
Key_Enter = 10,
Key_Esc = 27,
Key_W = 119,
Key_S = 115
};

public:

EventLoop(Window *);

void runMenu();

void auth();

void play(Player,Player);

void options();

void records();

void sign_in();

};

#endif // EVENT_LOOP_HPP

```

//EventLoop.cpp

```

#include "EventLoop.hpp"

#include "../Utils/FileSystem/FileSystem.hpp"

#include "../GameObjects/Board.hpp"

#include <string>

#include <ctime>

EventLoop::EventLoop(Window *win) : window(win) {}

void EventLoop::runMenu()

{

int selected = 0;

int ch;

while ((ch = window->input()) != Key_Esc && !exit)

{

window->cleanWindow();

```

```

window->drawText(1, 67, ">>>>>>>MENU>>>>>>>>");

switch (ch)
{
case Event::Key_Up:
selected--;
if (selected < 0)
selected = 3;
break;
case Event::Key_Down:
selected++;
if (selected > 3)
selected = 0;
break;
case Event::Key_Enter:
switch (selected)
{
case 0:
auth();
break;
case 1:
options();
break;
case 2:
records();
break;
case 3:
exit = true;
break;
}
break;
}

```

```

}
window->drawMenu(selected);
}
window->cleanWindow();
}
void EventLoop::auth()
{
int selected = 0;
int ch;
bool backToMenu = false;
while ((ch = window->input()) != Key_Esc && !backToMenu)
{
window->cleanWindow();
switch (ch)
{
case Event::Key_Up:
selected--;
if (selected < 0)
selected = 1;
break;
case Event::Key_Down:
selected++;
if (selected > 1)
selected = 0;
break;
case Event::Key_Enter:
switch (selected)
{
case 0:
sign_in();

```

```

break;

case 1:

backToMenu = true;

break;

}

}

window->drawAuth(selected);

}

window->cleanWindow();

}

void EventLoop::play(Player player1, Player player2)
{
int ch;

Board board(player1, player2);

while ((ch = window->input()) != Key_Esc)
{
window->cleanWindow();

window->drawText(1, 67, ">>>>>>>>SPACE RACE>>>>>>>>");

switch (ch)
{
case Event::Key_Up:

board.movePlayer1(Direction::Up);

break;

case Event::Key_Down:

board.movePlayer1(Direction::Down);

break;

case Event::Key_W:

board.movePlayer2(Direction::Up);

break;

case Event::Key_S:

```

```

board.movePlayer2(Direction::Down);

break;

}

board.update();

int t = std::diffTime(time(0), board.startTime);

t = gameDelay - t;

if (t <= 0)

window->drawText(2, 67, "YOUR TIME IS UP");

window->drawTime(48, 3, t);

window->drawText(48, 66, board.getPlayer1().getName());

window->drawScore(48, 78, board.getPlayer1().getScore());

window->drawText(48, 80, "|");

window->drawText(48, 82, board.getPlayer2().getName());

window->drawScore(48, 94, board.getPlayer2().getScore());

window->drawBoard(board.getMatrix());

FileSystem::save_points(board.getPlayer2().getScore());

FileSystem::save_points(board.getPlayer1().getScore());

}

window->cleanWindow();

}

void EventLoop::options()

{

int selected = 0;

int ch;

while ((ch = window->input()) != Key_Esc)

{

window->cleanWindow();

window->drawText(1, 67, ">>>>>>>OPTIONS>>>>>>>>");

switch (ch)

{

```

```

case Event::Key_Up:
selected--;
if (selected < 0)
selected = 1;
break;
case Event::Key_Down:
selected++;
if (selected > 1)
selected = 0;
break;
case Event::Key_Enter:
switch (selected)
{
case 0:
gameDelay = 60;
sign_in();
break;
case 1:
gameDelay = 120;
sign_in();
break;
}
}
window->drawTimeSet(selected);
}
window->cleanWindow();
}
void EventLoop::records()
{
int ch;

```



```

while ((ch = window->input()) != Key_Esc)
{
window->cleanWindow();

window->drawText(1, 67, ">>>>>>>RECORDS>>>>>>>>");

window->drawText(2, 20, FileSystem::read_registration());
}

window->cleanWindow();
}

void EventLoop::sign_in()
{
Player player1;
Player player2;

char ch;

window->drawText(1, 67, ">>>>>>>SIGN IN>>>>>>>>");

window->drawText(25, 70, "PLAYER 1'S NAME: ");

std::string playerName = "";

while ((ch = window->input()) != Event::Key_Enter)
{
if (ch == -1)

continue;

playerName.push_back(ch);

window->drawText(25, 89, playerName);

//FileSystem::save_name(board.getPlayer1Name());
}

player1.setName(playerName);

playerName = "";

window->drawText(28, 70, "PLAYER 2'S NAME: ");

while ((ch = window->input()) != Event::Key_Enter)
{
if (ch == -1)

```

```

continue;

playerName.push_back(ch);

window->drawText(28, 89, playerName);

//FileSystem::save_name(getPlayer2().getName());

}

player2.setName(playerName);

play(player1, player2);

}

```

//FileSystem.hpp

```

#ifndef FILE_SYSTEM_HPP
#define FILE_SYSTEM_HPP

#include <string>
#include <fstream>

namespace FileSystem
{
    void save_name(std::string array);
    void save_points(int point);
    std::string read_registration();
    const char *convert_int_to_char(int integer);
} // namespace FileSystem

#endif // FILE_SYSTEM_HPP

```

//FileSystem.cpp

```

#include "FileSystem.hpp"

void FileSystem::save_name(std::string array)
{
    std::fstream file;

    file.open("registration_list.txt", std::ios::in);

    if (!file)
    {
        return;
    }
}

```

```

    }
    else
    {
        for (int i = 0; i < (int)array.size(); ++i)
        {
            file << array[i] << std::endl;
        }
        file.close();
    }
}

void FileSystem::save_points(int point)
{
    std::fstream file;
    file.open("registration_list.txt", std::ios::app);
    if (!file)
    {
        return;
    }
    else
    {
        file << point << std::endl;
    }
    file.close();
}

std::string FileSystem::read_registration()
{
    std::string registration;
    std::fstream file;
    file.open("registration_list.txt", std::ios::out);

```

```

if (!file)
{
return "";
}
else
{
for (int i = 0; i < (int)registration.size(); ++i)
{
file >> registration[i];
}
file.close();
}
return registration;
}

const char *FileSystem::convert_int_to_char(int integer)
{
std::string temp_str = std::__cxx11::to_string(integer);
char const *integers_array = temp_str.c_str();
return integers_array;
}

```

//GameObjects.hpp

```

#ifndef GAME_OBJECT_HPP
#define GAME_OBJECT_HPP

#include <vector>

enum Direction
{
Left,
Right,
Up,
Down,

```

None

```
};  
  
class GameObjects  
{  
protected:  
    int x;  
    int y;  
  
    using Matrix = std::vector<std::vector<char>>>;  
  
public:  
    GameObjects() {}  
    virtual ~GameObjects() {}  
    virtual void move(Direction, const Matrix &) = 0;  
    virtual char getSymbol() const = 0;  
    int getx() const { return x; }  
    int gety() const { return y; }  
    void setxy(int x, int y)  
    {  
        this->x = x;  
        this->y = y;  
    }  
    void setx(int x) { this->x = x; }  
};  
  
#endif // GAME_OBJECT_HPP
```

//Spaceships.hpp

```
#ifndef SPACESHIPS_HPP  
#define SPACESHIPS_HPP  
#include "GameObjects.hpp"  
  
class Spaceships : public GameObjects  
{  
public:
```

```

Spaceships() {}

~Spaceships() {}

public:

void move(Direction, const Matrix &);

char getSymbol() const;

};

#endif // SPACESHIPS_HPP

//Spaceships.cpp

#include "Spaceships.hpp"

void Spaceships::move(Direction dir, const Matrix &matrix)

{
    switch (dir)
    {
        case Direction::Up:
            if (x - 1 < 0)
            {
                x = matrix.size() - 1;
            }
            else
            {
                --x;
            }
            break;

        case Direction::Down:
            if (x + 1 < (int)matrix.size())
            {
                ++x;
            }
            break;

        default:
            break;
    }
}

char Spaceships::getSymbol() const
{
    return '^';
}

```

```
}
```

//Stars.hpp

```
#ifndef STARS_HPP
#define STARS_HPP
#include "GameObjects.hpp"
class Stars : public GameObjects
{
private:
    Direction dir;
public:
    Stars(Direction);
    ~Stars() {}
    void move(Direction, const Matrix &);
    char getSymbol() const;
};
#endif // STARS_HPP
```

//Stars.cpp

```
#include "Stars.hpp"
#include <time.h>
#include <stdlib.h>
Stars::Stars(Direction dir) : dir(dir) {}
void Stars::move(Direction, const Matrix &matrix)
{
    switch (dir)
    {
    case Direction::Right:
        if (y + 1 >= (int)matrix[0].size())
            y = 0;
        else
            ++y;
    }
```

```

break;

case Direction::Left:
if (y - 1 <= 0)
y = matrix[0].size() - 1;
else
--y;
break;

default:
break;
}
}

char Stars::getSymbol() const
{
return '*';
}

```

//Players.hpp

```

#ifndef PLAYER_HPP
#define PLAYER_HPP
#include <string>

class Player
{
private:
std::string name;
int score = 0;
public:
Player() {}

std::string getName() const;
int getScore() const;
void setName(std::string name);
void setScore(int score);

```



```

};

#endif // PLAYER_HPP

//Players.cpp

#include "Player.hpp"

void Player::setName(std::string name)
{
    this->name = name;
}

void Player::setScore(int score)
{
    this->score = score;
}

std::string Player::getName() const
{
    return name;
}

int Player::getScore() const
{
    return score;
}

//run.sh

#!/bin/sh

g++ -Wall -g main.cpp $(find SpaceRace -type f -iregex ".*\.cpp") -lncurses

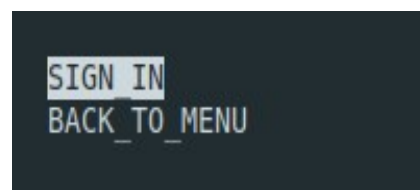
```

Արտաքին դիզայնը

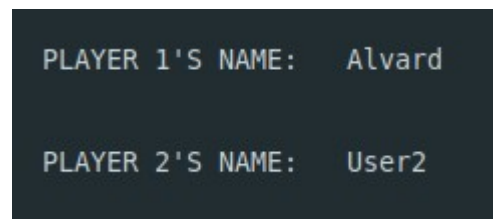
Menu



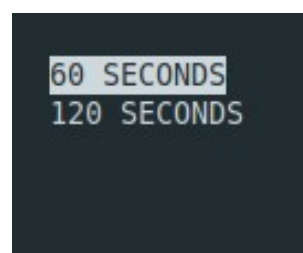
Play



Sign in

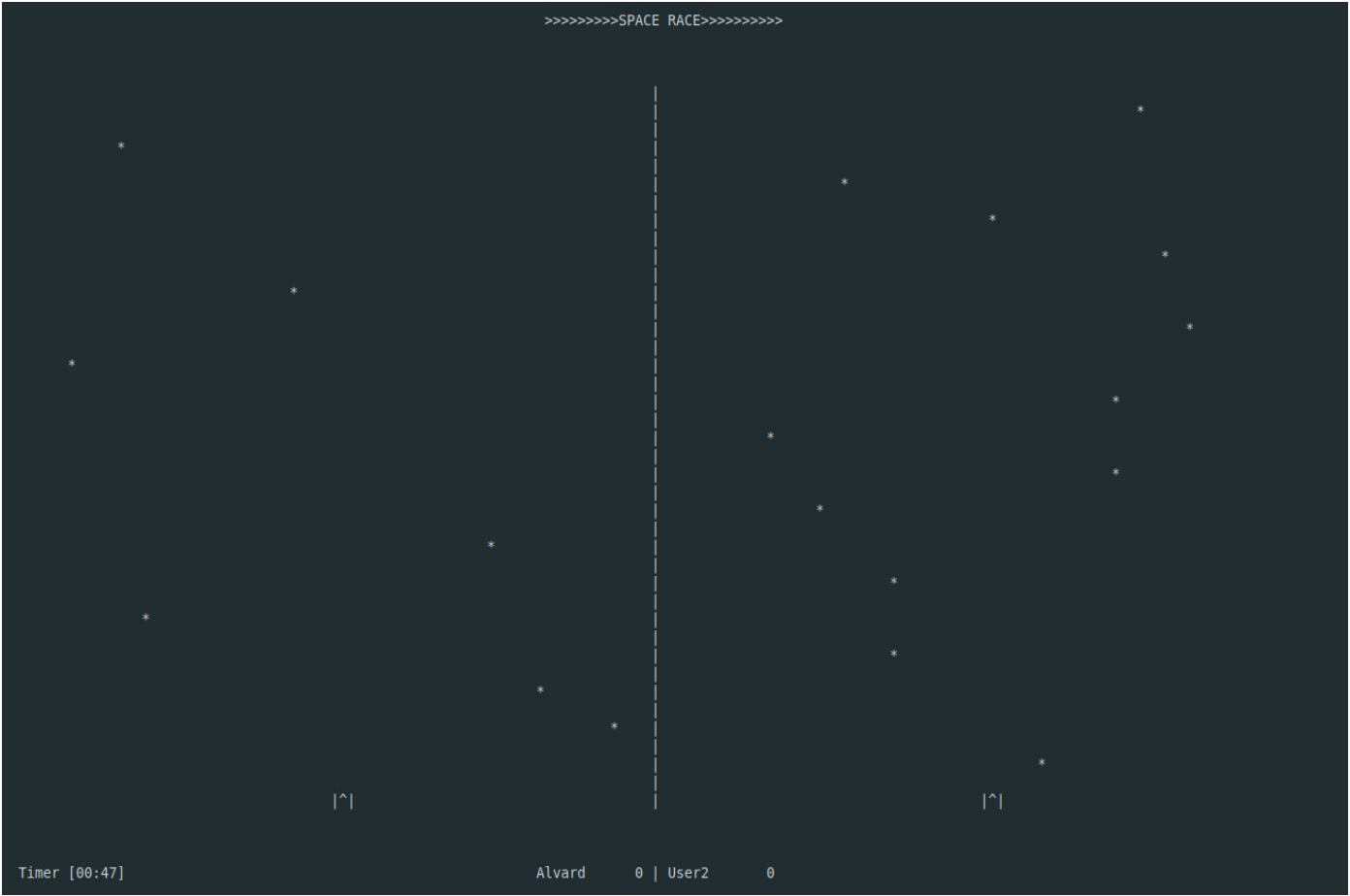


Options



Արտաքին դիզայնը

Խաղը



Եզրակացություն

Կուրսային աշխատանքի կատարման ընթացքում սովորեցի թե ինչպես են նախագծում մեծ պրոյեկտները: Ուսումնասիրեցի Ncurses գրադարանը , որի օգնությամբ ստեղծել եմ կուրսային աշխատանքս: Եվ ավելի խորությամբ ուսումնասիրեցի C++ լեզուն , որը ապագայում դեռ շատ պետք կգա: Սովորեցի նոր պարադիգմեր եւ դիզայնների ձեւանմուշներ: Օգտագործեցի C++ լեզվի vector, string, iostream, fstream, time, գրադարանները: Խաղի ծրագիրը իրականացնելու համար գրեցի shell որի շնորհիվ sh run.sh գրելով ապա, գրելով ./a.out տերմինալում կաշխատի և կբացի խաղի menu-ն:

Գրականության ցանկ

C++ լեզվի հիմնական գիտելիքներ

Ռաֆայել Շահոյանի դասախոսություններ

Bjarne Stroustrup – The C++ Programming Language

ncurses գրադարան

<http://www.cems.uwe.ac.uk/~irjohnso/courses/coursenotes/ufs001/NCURSES-Programming-HOWTO.pdf>

C++ լեզվի այլ գրադարանների մասին տեղեկություններ

<https://www.geeksforgeeks.org/c-plus-plus/?ref=ghm>

<https://en.cppreference.com/w/cpp>